

PROJE GEREKSİNİMLERİ DOKÜMANI (PRD) — v2

SentinelWatch — Faz 2: Dinamik Kural Motoru ve Alarm Geçmişi

Hedef Kitle: Karahan (Backend) & Yağız (Frontend)

Süre: 1 Hafta — Tarih: Temmuz 2026

1. Bu Fazın Amacı

Mevcut SentinelWatch sistemi çalışıyor: olaylar üretiliyor, ekranda görünüyor, birkaç sabit kontrol alarm üretebiliyor. Bu faz, var olan sistemi sıfırdan yeniden yazmak yerine üzerine iki yeni yetenek ekliyor: alarmların kalıcı olarak saklanması ve kuralların kod değiştirmeden, panelden yönetilebilir hale gelmesi.

Gerçek yazılım ekiplerinde de işler tam olarak böyle ilerler — çalışan bir sistem nadiren sıfırdan yazılır, kademeli olarak genişletilir. Bu fazda amaç, var olan kodu bozmadan üzerine sağlam, iyi düşünülmüş iki özellik eklemektir. Mevcut kod tabanı korunacak; tabloya yeni alanlar ve yeni tablolar eklenecek, API'ye yeni endpoint'ler eklenecek.

Neyin Değiştigi — Özet

Konu	Şu Anki Durum	Bu Fazdan Sonra
Alarmlar	İstek geldiğinde anlık hesaplanır, saklanmaz	Alerts tablosunda kalıcı olarak tutulur, geçmişi görüntülenebilir
Kurallar	Kod içinde sabit 2-3 fonksiyon	Rules tablosunda saklanır, panelden eklenip kapatılabilir
Veri iletimi	5 saniyede bir polling	Değişmiyor (bu fazın kapsamı dışında)
AI katmanı	Yok	Değişmiyor (bu fazın kapsamı dışında)

WebSocket'e geçiş ve LLM katmanı bilinçli olarak bu fazın dışında bırakılmıştır. Bunlar ayrı birer eksen olduğu için, mevcut hafta içinde iki kişiye de dengeli ve bitirilebilir bir yük dağıtmak adına kapsam dışında tutulmuştur.

2. Rol Dağılımı ve Görevler

2.1 Karahan — Backend: Kalıcı Alarm ve Dinamik Kural Motoru

- Rules tablosunun eklenmesi (bkz. Bölüm 3.1) ve ilişkili migration'ın yazılması.
- Alerts tablosunun eklenmesi (bkz. Bölüm 3.2) — alarm artık anlık hesaplanmıyor, olduğu anda kaydediliyor.
- Mevcut 2-3 sabit fonksiyonun (check_brute_force vb.), Rules tablosundan okuyan tek bir genel değerlendirme fonksiyonuna dönüştürülmesi: aynı mantık, ama artık eşik/pencere/olay tipi değerleri kod içinde değil veritabanında.
- Yeni endpoint'ler: kural oluşturma, listeleme, aktif/pasif yapma (bkz. Bölüm 4).
- Alarm geçmişi için filtreleme ve 'incelendi' olarak işaretleme endpoint'i.
- Var olan /api/events ve mevcut alarm mantığının bu değişiklikten etkilenmediğinden emin olunması (geriye dönük uyumluluk).

2.2 Yağız — Frontend: Kural Yönetimi ve Alarm Geçmişi Ekranları

- 'Kural Ekle' formu: olay tipi, eşik sayısı, zaman penceresi, önem derecesi alanlarını içeren basit bir form.
- Kural listesi paneli: tanımlı kuralların gösterilmesi, her birinin aktif/pasif yapılabilmesi (toggle).
- Alarm geçmişi ekranı: artık kalıcı olan alarmların listelenmesi, önem derecesine ve tarihe göre filtrelenmesi.
- Bir alarmı 'incelendi' olarak işaretleme butonu ve buna göre görsel durum değişikliği (ör. soluklaşan satır).
- Var olan canlı akış ve grafik ekranlarının bozulmadan çalışmaya devam etmesi (regresyon yok).

3. Veritabanı Değişiklikleri

3.1 Rules Tablosu (Yeni)

Alan	Tip	Açıklama
id	integer (PK)	Kural kimliği
name	string	Örn: 'Brute Force Girişi'
event_type	string	İzlenecek olay tipi (ör. LOGIN_FAILED)
threshold_count	integer	Tetiklenme için gereken olay sayısı
time_window_seconds	integer	Değerlendirme zaman penceresi
severity	string	low / medium / high
is_active	boolean	Kuralın devrede olup olmadığı

3.2 Alerts Tablosu (Yeni)

Alan	Tip	Açıklama
id	integer (PK)	Alarm kimliği
rule_id	integer (FK -> Rules)	Hangi kuralın tetiklendiği
timestamp	datetime	Alarmin oluşturulma zamanı
source_ip	string	İlgili IP (varsa)
description	string	İnsan tarafından okunabilir açıklama
is_resolved	boolean	İncelendi/kapatıldı mı

Events tablosu değişiyor. Bu faz sadece iki yeni tablo ekliyor; mevcut yapı korunuyor.

4. API Sözleşmesi Eklemeleri

Metod	Endpoint	Açıklama
GET	/api/rules	Tanımlı kuralları listeler
POST	/api/rules	Yeni kural oluşturur
PATCH	/api/rules/{id}	Kuralı aktif/pasif yapar

GET	/api/alerts/history	Kalıcı alarm geçmişini filtreli listeler (severity, isResolved, from, to)
PATCH	/api/alerts/{id}/resolve	Bir alarmı incelendi olarak işaretler

4.1 POST /api/rules — İstek Örneği

```
{
  "name": "Anormal Trafik Artisi",
  "eventType": "REQUEST",
  "thresholdCount": 100,
  "timeWindowSeconds": 60,
  "severity": "medium"
}
```

4.2 GET /api/alerts/history — Yanıt Örneği

```
[
  {
    "id": 41,
    "ruleName": "Brute Force Girisi",
    "timestamp": "2026-07-13T09:12:00Z",
    "sourceIp": "185.23.11.4",
    "description": "185.23.11.4 adresinden 5 dakikada 6 basarisiz giris",
    "severity": "high",
    "isResolved": false
  }
]
```

5. 1 Haftalık Yol Haritası

Gün 1 — Sözleşme ve Şema (Birlikte)

- Bölüm 3 ve 4'teki tablo şeması ve endpoint'ler üzerinde birlikte mutabakata varılır.
- Karahan: Rules ve Alerts tablolarının migration'ı yazılır.
- Yağız: Kural formu ve alarm geçmişi ekranlarının mock veriyle wireframe'i hazırlanır.

Gün 2-4 — Paralel Geliştirme

- Karahan: Sabit fonksiyonların Rules tablosundan okuyan genel bir değerlendirme fonksiyonuna dönüştürülmesi; yeni alarm artık Alerts tablosuna yazılıyor. /api/rules ve /api/alerts/history endpoint'lerinin tamamlanması.
- Yağız: Gerçek /api/rules ile kural formunun ve listesinin bağlanması; gerçek /api/alerts/history ile geçmiş ekranının bağlanması, filtrelerin çalışır hale gelmesi.

Gün 5-7 — Entegrasyon, Test ve Kapanış

- Karahan ve Yağız (Birlikte): Uçtan uca test — panelden yeni bir kural eklenir, bu kuralı tetikleyecek sahte olaylar üretilir (bkz. not aşağıda), alarmın geçmişte doğru görüldüğü ve 'incelendi' işaretlenebildiği doğrulanır.
- Var olan canlı akış ve grafik ekranlarının bu değişikliklerden etkilenmediği kontrol edilir (regresyon testi).
- README.md, yeni tablolar ve yeni endpoint'leri de kapsayacak şekilde güncellenir.

Test için pratik yol: backend'e basit bir '/debug/inject-event' endpoint'i eklemeleri önerilir — bu sayede rastgele olay üretimini beklemek yerine, istedikleri anda istedikleri tipte sahte olay göndererek yeni eklenen kuralı anında tetikleyip sonucu gözlemleyebilirler.

6. Başarı Kriterleri (Definition of Done — v2)

1. Panelden yeni bir kural eklenebiliyor ve bu kural, ilgili koşul oluştuğunda gerçekten alarm üretiyor (kod değişikliği gerekmiyor).
2. Üretilen alarmlar Alerts tablosunda kalıcı olarak saklanıyor ve geçmiş ekranından görüntülenebiliyor.
3. Bir alarm 'incelendi' olarak işaretlenebiliyor ve bu durum arayüze yansıyor.
4. Var olan canlı akış ve grafik ekranları, bu değişikliklerden önce olduğu gibi sorunsuz çalışmaya devam ediyor.
5. Geçersiz kural tanımı (ör. eksik alan) gönderildiğinde backend çökmüyor, uygun hata dönüyor.
6. README, yeni tablo ve endpoint'leri kapsayacak şekilde güncellenmiş durumda.

“Bu fazda asıl beceri, sıfırdan bir şey yazmak değil, çalışan bir sistemi bozmadan büyütme. Gerçek mühendislikte en çok karşılaştığınız durum tam olarak budur. İyi çalışmalar.”